

SISTEMAS DE COMPUTADORES – 2024/2025
Exame Época de Recurso

Versão A

Apenas autorizada a consulta da folha oficial
Duração: 2h

Nome: _____ Nº _____

Grupo I
(8/20 valores)

NOTAS:

- 1. Em todas as questões deverá assinalar apenas uma resposta.**
- 2. Se a resposta assinalada for incorreta sofrerá uma penalização de 1/3 da cotação da pergunta.**
- 3. Devem ser entregues todas as folhas do exame.**

1 – Ao invocar um serviço do sistema operativo por meio de uma *system call*, que mecanismo efetua a transição de *user space* para *kernel space*?

- a) Uma interrupção de hardware
- b) Uma exceção síncrona gerada por software (*trap*)
- c) Um sinal assíncrono enviado pelo *kernel* ao processo
- d) Uma normal invocação de uma função em *user space*

2 – Em Linux, após a invocação da *system call fork()*, qual das seguintes afirmações sobre o espaço de endereçamento do processo filho é verdadeira?

- a) O processo filho partilha fisicamente toda a memória com o processo pai
- b) O espaço de endereçamento do filho é completamente novo, sem qualquer relação com o do processo pai
- c) O processo filho tem um espaço de endereçamento distinto, mas mapeado para as mesmas páginas físicas, sendo separada apenas após uma operação de escrita
- d) O processo filho utiliza exclusivamente a memória do *kernel* até invocar uma função *exec*

3 – Como é que um sistema operativo com escalonamento preemptivo assegura a partilha de CPU entre processos?

- a) Através da invocação voluntária de uma função, por parte do processo em execução, no final de um bloco de instruções do programa
- b) Através de uma interrupção de um temporizador em hardware (*timer interrupt*) que força uma mudança de contexto periódica
- c) Através da análise periódica (*polling*) do estado dos processos por parte do sistema operativo
- d) Apenas quando um processo bloqueia em I/O ou termina, nunca de modo involuntário para o processo em execução

4 – Qual das seguintes afirmações descreve corretamente a principal responsabilidade do *dispatcher* num sistema operativo?

- a) Decidir que processo deve ser executado a seguir e por quanto tempo, com base em políticas de escalonamento
- b) Gerir a alocação de memória para novos processos e libertar a memória de processos terminados
- c) Otimizar o balanceamento de carga entre múltiplos núcleos de processamento para maximizar o débito computacional
- d) Realizar a comutação de contexto, regressar para o espaço de utilizador e transferir o controlo do CPU para o processo selecionado

5 – Qual das seguintes afirmações descreve corretamente a diferença entre *faults* e *traps*, no contexto das exceções síncronas?

- a) *Faults* são geradas por interrupções externas; *traps* são geradas por chamadas ao sistema
- b) *Traps* devolvem o controlo à instrução atual; *faults* nunca permitem retomar a execução da instrução atual
- c) *Faults* ocorrem devido a erros detetáveis e (potencialmente) recuperáveis; *traps* são exceções intencionais e devolvem o controlo à instrução seguinte
- d) Ambas representam falhas fatais de hardware, sem possibilidade de recuperação

6 – Qual das seguintes afirmações descreve corretamente a inanição (*starvation*) em sistemas concorrentes?

- a) É uma condição em que um processo tenta repetidamente uma ação que falha, mas não está bloqueado
- b) Ocorre quando um processo é continuamente impedido de aceder a um recurso necessário, impedindo-o de executar
- c) É uma situação em que dois ou mais processos estão bloqueados indefinidamente, cada um esperando por um recurso detido pelo outro
- d) Resulta da reordenação de instruções por compiladores e processadores, levando a resultados inconsistentes

7 – No contexto do problema dos Leitores-Escritores, qual é o principal compromisso entre a primeira variante (favorece leitores) e a segunda variante (favorece escritores) do algoritmo?

- a) A primeira variante pode levar à inanição (*starvation*) de escritores, enquanto a segunda variante pode levar à inanição de leitores
- b) A primeira variante é mais complexa de implementar, enquanto a segunda variante é mais simples
- c) A primeira variante garante que nenhum escritor será bloqueado, enquanto a segunda variante garante que nenhum leitor será bloqueado
- d) A primeira variante permite um maior grau de concorrência nas operações de leitura e escrita, enquanto a segunda variante impõe um acesso estritamente sequencial

8 – Qual das seguintes **não** é uma das quatro condições necessárias para a ocorrência de bloqueio (*deadlock*)?

- a) Exclusão mútua
- b) Espera por retenção (*hold and wait*)
- c) Preempção obrigatória
- d) Espera circular

9 – Qual das seguintes afirmações compara corretamente os algoritmos *Shortest Job First* (SJF) e *Shortest Remaining Time First* (SRTF)?

- a) O SJF é uma versão preemptiva do SRTF, otimizando o tempo de resposta em sistemas interativos
- b) O SRTF é provadamente ótimo para o tempo de retorno médio quando todos os processos estão disponíveis simultaneamente, ao contrário do SJF
- c) SJF e SRTF podem levar à inanição (*starvation*) de processos com longos tempos de execução, mas o SRTF é mais complexo devido à necessidade de monitorização contínua
- d) O SJF não necessita de estimativas de tempo de execução, enquanto o SRTF exige previsões precisas para funcionar eficazmente

10 – Considere um sistema operativo que utiliza escalonamento *Round-Robin* (RR). Qual das seguintes afirmações sobre o algoritmo RR e o seu *time slice* é correta?

- a) Um *time slice* muito longo no RR resulta numa melhor performance interativa, pois os processos têm mais tempo para completar as suas tarefas
- b) O RR é um algoritmo não-preemptivo que garante que os processos *CPU-bound* não monopolizem a CPU
- c) Se o *time slice* for excessivamente curto, o débito computacional (*throughput*) do sistema pode sofrer significativamente devido ao aumento do *overhead* da comutação de contexto
- d) O RR elimina completamente o *overhead* da comutação de contexto, tornando-o o algoritmo mais eficiente em todas as situações

Grupo II (12/20 valores)

Versão A

NOTAS:

1. Responder cada questão numa folha separada, devidamente identificada.
2. Indicar a versão do exame em cada uma das folhas.
3. Não é necessário indicar o nome das bibliotecas usadas na resolução.

[5 valores] 1) Implemente um programa em C que simule um sistema concorrente de processamento de encomendas. O processo principal (coordenador) lê do *stdin* uma sequência de M pedidos de encomenda, cada um representado por um número inteiro positivo (ex., o número de unidades de um artigo). Estes pedidos devem ser distribuídos equitativamente por N processos filho (trabalhadores), criados previamente.

Cada trabalhador recebe os pedidos que lhe forem atribuídos através de um *pipe*, calcula o custo total de cada encomenda (ex., multiplicando o número de unidades por um valor fixo; pode considerar que todos os produtos têm o mesmo valor fixo), e envia os resultados de volta para o processo coordenador, que os imprime pela ordem em que os pedidos foram introduzidos.

A sua solução deve:

- Criar o número adequado de processos e *pipes* para garantir **execução concorrente**;
- Assegurar a correcta associação de cada pedido ao trabalhador responsável;
- **Preservar a ordem de apresentação dos resultados** relativamente à entrada.

[7 valores] 2) Implemente um programa em C que simule um **semáforo rodoviário inteligente**, que alterna o direito de passagem entre **carros** e **peões**, usando **threads** e **sincronização**.

O programa deve criar **duas threads geradoras de tráfego** (uma simula a chegada de **carros** e outra simula a chegada de **peões**) e **uma thread controladora do semáforo** (que regula alternadamente quem pode atravessar, carros ou peões).

O comportamento esperado é o seguinte:

- Quando o semáforo está verde para peões, apenas peões podem atravessar; o mesmo se aplica aos carros, que só podem atravessar quando o semáforo estiver verde para os carros;
- Cada passagem demora 1 segundo;
- O semáforo **muda de estado** (de “verde para carros” para “verde para peões”, ou vice-versa) após permitir a travessia de 5 entidades (carros ou peões);
- Carros e peões **bloqueiam de forma passiva até o semáforo estar verde para o seu tipo**;

Assuma que cada *thread* geradora de tráfego gera **30 entidades** (carros ou peões). Para resolver este exercício (e apenas para esse efeito) pode considerar que dentro de uma *thread* pode usar a função `printf()` de uma forma segura.

O *output* do programa deve incluir mensagens como:

```
[PEÃO] Peão 1 a aguardar
[CARRO] Veículo 5 atravessou
[SEMAFORO] Mudança para peões
[PEÃO] Peão 1 atravessou
[CARRO] Veículo 6 a aguardar
```